



NPTEL Online Certification Course
Applied Accelerated Artificial Architecture
Assignment 3 : Detailed Solution
Indian Institute of Technology Guwahati

Question 1:

What is the final shape of the tensor after performing the following tensor operations sequentially?

```
import torch
# Initial tensor
x = torch.randn(2, 1, 3)
```

```
# Sequential tensor transformations
y = x.unsqueeze(-1)
z = y.squeeze(dim=0)
w = z.squeeze(dim=-1)
print(w.shape)
```

- A) `torch.Size([2, 1, 3])`
- B) `torch.Size([1, 2, 3, -1])`
- C) `torch.Size([2, 1, 3, 1])`
- D) `torch.Size([1, 2, 3])`

Correct Answer: A) `torch.Size([2, 1, 3])`

Explanation: `unsqueeze(-1)` adds a dimension at the end to make shape `(2, 1, 3, 1)`, but `squeeze(dim=0)` does nothing because dimension 0 has size 2 (not 1). Finally, `squeeze(dim=-1)` removes the trailing size-1 dimension, returning the tensor back to its original shape `(2, 1, 3)`.

Question 2

What is the primary technical reason PyTorch recommends avoiding in-place operations on tensors participating in an autograd graph?

- A) Always
- B) Sometimes
- C) Never
- D) None of them

Correct Answer: C) Never

Explanation: In-place operations directly modify tensor memory, which destroys the original values needed by autograd to compute derivatives during the backward pass. PyTorch disallows or raises errors on these operations to maintain graph integrity and prevent silent, incorrect gradient calculations.

Question 3 :

According to PyTorch's broadcasting rules, how are dimensions compared between two tensors?

- A) From left to right, missing dimensions are padded with zeros.
- B) From right to left (trailing dimensions first), where two dimensions are compatible if they are equal or one of them is 1.
- C) From the largest dimension to the smallest dimension.
- D) PyTorch randomly aligns dimensions until it finds a valid combination.

Correct Answer: B) From right to left (trailing dimensions first), where two dimensions are compatible if they are equal or one of them is 1.

Explanation: Broadcasting aligns tensor shapes by comparing their dimension sizes starting from the rightmost (trailing) dimension and moving leftward. Two dimensions are compatible only if they are equal or if one of them has a size of 1, which gets expanded to match the other.

Question 4 :

What happens to the memory usage of your program when you set `requires_grad=True` for many tensors?

- A) Memory remains exactly same
- B) Python stores the computational graph for backward propagation
- C) Memory usage decreases
- D) Memory cleared automatically

Correct Answer: B) Python stores the computational graph for backward propagation

Explanation: Setting `requires_grad=True` instructs PyTorch to construct an execution graph that tracks every intermediate tensor operation performed. This graph holds references to internal activation state tensors in memory so gradients can be properly computed during `.backward()`.

Question 5

Which segment / line would cause this PyTorch module implementation to fail and throw an error?

```
import torch
import torch.nn as nn

class MyLayer(nn.Module):
    def __init__(self):
        super().__init__()      # ALWAYS call this
        self.linear = nn.Linear(4, 3) # creates W and b
        self.relu = nn.ReLU

    def forward(self, x):
        x = self.linear(x)      # calls linear.forward(x)
        return self.relu(x)

# Instantiate the layer
layer = MyLayer()

# Pass input tensor through the layer
out = layer(torch.randn(2, 4))  # calls Layer.forward()

print(out)
```

- A) `self.relu = nn.ReLU`
- B) `def forward(self, x):`
- C) `x = self.linear(x)`
- D) `return self.relu(x)`

Correct Answer: A) `self.relu = nn.ReLU`

Explanation: The provided snippet is fully syntactically correct and outputs a valid tensor of shape (2, 3) without raising any exceptions. It only fails if option A (`self.relu = nn.ReLU` without parentheses) is substituted into `__init__`, making `self.relu` an uninstantiated class rather than a callable module.

Question 6

Which of the following correctly describes the flow of execution for a single forward pass in **DynamicNet**?

```
import torch
import torch.nn as nn
class DynamicNet(nn.Module):
    def __init__(self, d):
        super().__init__()
        # Dynamic architecture combining Linear and ReLU modules
        self.layers = nn.ModuleList()
        for _ in range(2):
            self.layers.append(nn.Linear(d, d))
            self.layers.append(nn.ReLU())
    def forward(self, x):
        for layer in self.layers:
            x = layer(x)
        return x

model = DynamicNet(d=4)
```

- A) Linear(4, 4) -> Linear(4, 4) -> ReLU() -> ReLU()
- B) ReLU() -> Linear(4, 4) -> ReLU() -> Linear(4, 4)
- C) Linear(4, 4) -> ReLU() -> Linear(4, 4) -> ReLU()
- D) Linear(4, 4) and ReLU() applied in parallel across two separate streams, then merged.

Correct Answer: C) Linear(4, 4) -> ReLU() -> Linear(4, 4) -> ReLU()

Explanation: The `__init__` loop alternates appending a `Linear(d, d)` layer followed immediately by a `ReLU()` activation layer twice into `nn.ModuleList`. In `forward()`,

iterating over `self.layers` executes these four sub-modules strictly sequentially in the exact order they were added.

Question 7

Which of the following statements is true regarding PyTorch loss functions (such as `CrossEntropyLoss`, `BCEWithLogitsLoss`, `MSELoss`, `L1Loss`, `HuberLoss`)?

- A) `CrossEntropyLoss` requires a Softmax layer in the model.
- B) `BCEWithLogitsLoss` requires integer target labels.
- C) `MSELoss` is robust against extreme data outliers.
- D) `CrossEntropyLoss` takes raw logits and integer labels.

Correct Answer: D) `CrossEntropyLoss` takes raw logits and integer labels.

Explanation: PyTorch's `CrossEntropyLoss` internally combines `LogSoftmax` and `NLLLoss` into a single numerically stable function, so models should output raw logits without a manual Softmax. It expects unnormalized continuous model outputs paired with target class indices formatted as integers.

Question 8

What is the purpose of calling `model.eval()` at the beginning of an evaluation loop?

- A) To clear previously computed gradients.
- B) To enable automatic mixed precision (AMP) during evaluation.
- C) To disable layers like dropout and batch normalization.
- D) To calculate gradients.

Correct Answer: C) To disable layers like dropout and batch normalization.

Explanation: Calling `model.eval()` switches the model into evaluation mode, altering the behavior of specific mechanisms like Dropout and Batch Normalization. It stops Dropout from randomly zeroing out activations and forces Batch Normalization to use running dataset statistics instead of batch statistics.

Question 9

Why is it crucial to call `optimizer.zero_grad()` in the training loop before computing the backward pass?

- A) To update the weight parameters and biases.
- B) To move the data to the target device.
- C) To compute the loss function.
- D) To reset the old gradient values from previous iterations.

Correct Answer: D) To reset the old gradient values from previous iterations.

Explanation: By default, PyTorch accumulates (adds together) gradients in `.grad` buffers during every backward pass rather than overwriting them. Calling `optimizer.zero_grad()` clears these stale values so old gradients from previous mini-batches do not corrupt current weight updates.

Question 10

In PyTorch training pipelines, what error occurs if data is not moved to the device (e.g., GPU) before executing the forward pass?

- A) Runtime Error
- B) Vanishing Gradient Error
- C) Zero Division Error
- D) Convergence Error

Correct Answer: A) Runtime Error

Explanation: PyTorch cannot perform mathematical operations on tensors that reside on different physical hardware devices (such as CPU memory versus GPU memory). Attempting a forward pass with mismatched device locations raises a `RuntimeError` declaring device mismatch.

Question 11

Which learning rate scheduling strategy is described as smoothly decaying the learning rate throughout the training process?

- A) Linear Warm-up only
- B) Cosine Annealing
- C) Static Scheduling
- D) Ignore Index Scheduling

Correct Answer: B) Cosine Annealing

Explanation: Cosine Annealing uses a cosine curve function to gradually decrease the learning rate from a high starting point down to a minimum target value over time. This smooth decay helps the optimizer explore the loss landscape early and settle into local minima gently near the end of training.

Question 12

When implementing a custom PyTorch dataset class, which two core methods must be explicitly defined to allow the PyTorch DataLoader to automatically process and fetch samples?

- A) `__init__` and `__forward__`
- B) `__len__` and `__getitem__`
- C) `__update__` and `__collate__`
- D) `__transform__` and `__prefetch__`

Correct Answer: B) `__len__` and `__getitem__`

Explanation: The `__len__` method must return the total number of items in the dataset so the DataLoader knows its batch bounds. The `__getitem__` method defines how to retrieve and preprocess a single sample and its corresponding label by index.

Question 13

Which parameter in the PyTorch DataLoader constructor page-locks CPU memory, enabling faster and direct DMA transfers to the GPU?

- A) `persistent_workers`
- B) `drop_last`
- C) `pin_memory`
- D) `prefetch_factor`

Correct Answer: C) `pin_memory`

Explanation: Setting `pin_memory=True` allocates host (CPU) memory in a page-locked region, preventing the operating system from swapping it to disk. This enables host-to-device Direct Memory Access (DMA) transfers, significantly speeding up data movement to CUDA GPUs.

Question 14

Which of the following is the correct implementation for the missing code block inside `__getitem__` to extract a single pair of input data and its corresponding label by index?

```
import torch

from torch.utils.data import Dataset

class TextDataset(Dataset):

    def __init__(self, data, labels):

        self.data = data

        self.labels = labels

    def __len__(self):

        return len(self.data)
```

```
def __getitem__(self, index):  
    # Missing Code Block  
    return item, label
```

- A) `item = self.data.get(index); label = self.labels.get(index)`
- B) `item = self.data[index]; label = self.labels[index]`
- C) `item = self.data.pop(index); label = self.labels.pop(index)`
- D) `item = self.data.next(); label = self.labels.next()`

Correct Answer: B) `item = self.data[index]; label = self.labels[index]`

Explanation: PyTorch Datasets access underlying sequences (like lists, arrays, or tensors) using standard Python indexing syntax `[index]`. Using `.get()`, `.pop()`, or `.next()` will raise `AttributeError` exceptions because standard indexable sequence types do not support those methods for index lookup.

Question 15

If the entire dataset cannot be divided evenly by the chosen batch size, which `DataLoader` argument should be set to "True" to discard the final smaller batch?

- A) `shuffle`
- B) `persistent_workers`
- C) `prefetch_factor`
- D) `drop_last`

Correct Answer: D) `drop_last`

Explanation: When a dataset size isn't perfectly divisible by the batch size, the final batch will contain fewer samples than expected, which can introduce noisy gradients. Setting `drop_last=True` causes the `DataLoader` to discard this uneven trailing batch to ensure uniform batch sizes throughout training.